Ders 15 Çalışma Özeti

Ders 15 Çalışma Özeti

Genel Konular

- Öncelikli kuyruk (priority queue) ve temel işlemleri: `insert`, `remove` (delete), `peek`.
- Öncelikli kuyruğun farklı veri yapılarıyla gerçeklenmesi ve karmaşıklık karşılaştırması.
- Öncelikli kuyruğun kullanım alanları.
- Binary heap ile ekleme ve çıkarma örneği.
- Geçen haftanın alıştırmaları:
  - İkili ağacın heap olup olmadığını kontrol eden kod.
  - Sırasız dizide k'ıncı en küçük elemanı bulma.
  - Min heap'i max heap'e dönüştürme.

Hocanın Özellikle Vurguladığı Kısımlar

- Öncelikli kuyruk yalnız heap ile değil, dizi, bağlı liste ve dengeli ikili arama ağacı ile de gerçeklenebilir.
- Heap ile gerçekleme işlemleri $O(\log n)$'e indirir ve en öncelikli elemana $O(1)$'de erişim sağlar. Bu yüzden genelde tercih edilir.
- Dengesiz bir ağaçla öncelikli kuyruk kurmanın faydası yoktur; dengeli (AVL gibi) ağaç gerekir.
- `peek` ile `remove` farklıdır: `peek` yalnız değeri okur, `remove` elemanı çıkarıp yapıyı yeniden düzenler.
- Heap'e ekleme yaparken bütün iç düğümleri heapify etmek gerekmez. Yalnız yeni elemanın aşağıdan yukarı (bottom-up) yerleşmesi yeterlidir. Hoca bunu bilerek önce yanlış anlatıp öğrencilerin fark etmesini bekledi.
- Diziyi heap'e çevirirken heapify, çocuğu olan son düğümden başlamalı ve köke kadar geriye gitmelidir.

Kısa Tekrar Notları

Gerçekleme	`insert`	`remove`	`peek`
Sırasız dizi	$O(1)$	$O(n)$	$O(n)$
Sıralı bağlı liste	$O(n)$	$O(1)$	$O(1)$
Binary heap	$O(\log n)$	$O(\log n)$	$O(1)$
Dengeli BST (AVL vb.)	$O(\log n)$	$O(\log n)$	Derste $O(1)$ denildi (bkz. ders sonrası notlar)

- Kullanım alanları:
  - İşletim sisteminde süreç yönetimi ve kesme (interrupt) işleme.
  - Dijkstra en kısa yol algoritması.
  - Huffman kodlaması.
  - Bankada öncelikli müşteriler, uçakta business/economy sınıfı.
- k'ıncı en küçük eleman: min heap kur, `extractMin`'i k kez çağır. Karmaşıklık $O(n+k\log n)$.
- Min heap'ten max heap'e: diziye doğrudan `buildMaxHeap` uygula.

Detaylı Açıklamalar

Farklı Yapılarla Öncelikli Kuyruk

- **Sırasız dizi:** Ekleme sona yapılır, $O(1)$. Çıkarmak ve `peek` için en yüksek öncelikli eleman tüm dizide aranır, $O(n)$.
- **Sıralı bağlı liste:** Ekleme sırasında doğru yer aranır, $O(n)$. En öncelikli eleman hep baştadır; çıkarma ve `peek` $O(1)$.
- **Dengeli BST:** Ekleme ve silme $O(\log n)$.
- **Yığın (stack):** Bir öğrenci önerdi. Hoca, yığınla verimli bir öncelikli kuyruk kurulamayacağını söyledi.

Binary Heap ile Ekleme ve Çıkarma

Başlangıç max heap'i: `9, 3, 5, 1, 4, 2` (sıradaki müşteri 9).

İşlem	Sonuç	Açıklama
`insert(11)`	9, 3, 5, 1, 4, 2, 11	11 son boş göze konur
	9, 3, 11, 1, 4, 2, 5	11 > 5, yer değiştirir
	11, 3, 9, 1, 4, 2, 5	11 > 9, köke çıkar; sıradaki müşteri artık 11
`remove()`	5, 3, 9, 1, 4, 2 | 11	11 son elemanla yer değiştirir, `size` bir azalır
	9, 3, 5, 1, 4, 2	Heapify: 9 yukarı çıkar; 5'in tek çocuğu 2 küçük olduğu için durur

- Bankada yeni gelen ve daha öncelikli bir müşteri (11), sırada bekleyen 9'un önüne geçer. 11 hizmet alınca sıra yeniden 9'a gelir.

Ekleme kodu (doğru hali):

```
void insert(int heap[], int *size, int value) {
    int i = (*size)++;
    heap[i] = value;
    while (i > 0 && heap[(i - 1) / 2] < heap[i]) {   /* 0 tabanlı dizi */
        swap(&heap[i], &heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}
```

- Derste gösterilen ilk kod, eklemeden sonra `size / 2 - 1`'den köke kadar tüm iç düğümleri heapify ediyordu.
- Hoca arada bunun gereksiz olduğunu açıkladı. Ağaç zaten heap olduğu için yalnız yeni elemanın yukarı taşınması yeterlidir.
- Tüm iç düğümleri gezen döngü yalnız sıfırdan heap kurarken ya da heap sort'ta kullanılır.

Silme (kökü çıkarma):

- Kök son elemanla yer değiştirir, `size` bir azalır.
- Kökten aşağı doğru heapify yapılır.

İkili Ağacın Heap Olup Olmadığını Kontrol Etme

Kod iki kontrol yapar:

```
struct Node { int key; struct Node *left, *right; };

int countNodes(struct Node *root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

```c
bool isCompleteUtil(struct Node *root, int index, int n) {
    if (root == NULL) return true;
    if (index >= n) return false;
    return isCompleteUtil(root->left, 2 * index + 1, n) &&
           isCompleteUtil(root->right, 2 * index + 2, n);
}

bool isHeapUtil(struct Node *root) {
    if (root->left == NULL && root->right == NULL) return true;
    if (root->right == NULL) return root->key >= root->left->key;
    if (root->key >= root->left->key && root->key >= root->right->key)
        return isHeapUtil(root->left) && isHeapUtil(root->right);
    return false;
}

bool isHeap(struct Node *root) {
    int n = countNodes(root);
    return isCompleteUtil(root, 0, n) && isHeapUtil(root);
}
```

- `countNodes`, ağaçtaki düğüm sayısını özyinelemeli sayar.
- `isCompleteUtil`, her düğüme dizideki gibi bir indeks verir. Complete ağaçta indeksler 0'dan $n-1$'e boşluksuz dolar. Bir indeks n'e ulaşırsa arada boşluk var demektir; ağaç complete değildir.
- `isHeapUtil`, her ebeveynin çocuklarından büyük olduğunu kontrol eder (max heap). Min heap için karşılaştırma ters çevrilir.
  - Çocuksuz düğüm heap'tir.
  - Yalnız sol çocuk varsa ebeveyn ondan büyük olmalıdır. Complete ağaçta yalnız sağ çocuk olamaz.
  - İki çocuk varsa ikisinden de büyük olmalı ve iki alt ağaç da heap olmalıdır.
- `bool` için `<stdbool.h>` kütüphanesi kullanılır.
- Derste örnek heap'in kökü değiştirilince ya da aradan bir düğüm çıkarılınca program "not a heap" yazdı.

k'ıncı En Küçük Eleman

Adımlar:

1. Tüm elemanlar bir min heap'e yerleştirilir.
2. `extractMin` k kez çağrılır.
3. k'ıncı çağrının döndürdüğü değer sonuçtur.

Örnek: `12, 3, 5, 7, 19`, $k = 3$.

Adım	Min heap
12, 3, 5, 7, 19 tek tek eklenir	3, 7, 5, 12, 19
1. `extractMin` → 3	5, 7, 19, 12
2. `extractMin` → 5	7, 12, 19
3. `extractMin` → 7	3. en küçük eleman 7

- Karmaşıklık $O(n + k \log n)$: heap kurmak $O(n)$, her çıkarma $O(\log n)$.
- Diziyi baştan sıralamak, basit bir sıralama ile $O(n^2)$'ye çıkabilirdi.

Min Heap'i Max Heap'e Dönüştürme

- Ek ağaç kullanmadan, dizinin kendisi üzerinde yapılır.
- Çocuğu olan son düğümden başlanır, köke kadar her düğüm için max heapify (yukarıdan aşağı) çağrılır.

Örnek: min heap `3, 5, 7, 12, 19` (0 tabanlı).

Adım	Düğüm	Sonuç
1	1 (değer 5), çocukları 12 ve 19	3, 19, 7, 12, 5
2	0 (değer 3), çocukları 19 ve 7	19, 3, 7, 12, 5
	3 aşağıda; çocukları 12 ve 5	19, 12, 7, 3, 5

- Derste daha büyük ikinci bir örnek de çözüldü. İlk denemede başlangıç düğümü bir eksik seçildi. Çocuğu olan bir düğüm (4 numaralı düğüm) atlandığı için sonuç max heap olmadı.
- Düzeltmede o düğümden başlandı; tüm iç düğümler sırayla heapify edilince ağaç max heap oldu.
- Ders: çocuğu olan her düğüme mutlaka bakılmalıdır.

Ders sonrası notlar

- **BST'de `peek`:** Hoca, dengeli BST'de en büyük elemanın en sağdaki düğüm olduğunu ve ona "doğrudan" erişildiğini söyleyerek `peek` için $O(1)$ dedi. En sağdaki düğüme ulaşmak için kökten sağa doğru inmek gerekir; bu $O(\log n)$'dir. $O(1)$ ancak en büyük düğümü gösteren ayrı bir işaretçi tutulursa (ve her ekleme/silmede güncellenirse) mümkündür.
- **Başlangıç düğümü:** 0 tabanlı dizide çocuğu olan son düğümün indeksi $\lfloor n/2 \rfloor - 1$'dir (yani $\lfloor (n-2)/2 \rfloor$). 1 tabanlı dizide $\lfloor n/2 \rfloor$'dir. Sohbette önerilen $\lfloor (n-1)/2 \rfloor$ de güvenlidir; en fazla bir yaprakta gereksiz bir heapify çağrısı yapar.
- **k'ıncı en küçük elemanda heap kurma:** $O(n)$ maliyet, diziye `buildMinHeap` uygulanırsa geçerlidir. Derste yapıldığı gibi elemanlar tek tek eklenirse kurma $O(n \log n)$ olur.
- **Banka örneğindeki başlangıç dizisi:** `9, 3, 5, 1, 4, 2` geçerli bir max heap değildir. 3'ün çocukları 1 ve 4'tür, 4 > 3 olduğu için kural bozulur. Ekleme ve çıkarma adımları yine tablodaki gibi işler, ama geçerli bir başlangıç için örneğin `9, 4, 5, 1, 3, 2` kullanılabilir.
- **`isCompleteUtil` açıklaması:** Hoca indeks n'i aşınca ağacın "dengeli olmadığını" söyledi. Kontrol edilen özellik denge değil, ağacın complete olmasıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.